

Мнемо код (80C186)	Описание	Формат на командата	Действие	Операнди	O	S	Z	A	P	C
Data Transfer Instructions: à Instructions in this group are provided to move data either between its internal registers or between an internal register and a storage location in memory.										
MOV	Move Data: Transfer a byte or a word of data from a source operand to a destination operand.	MOV dest, src	(dest) ← (src) No flags affected.	MOV mem, accum MOV accum, mem MOV reg, reg MOV reg, mem MOV mem, reg MOV reg, immed MOV mem, immed MOV seg-reg, reg16 MOV seg-reg, mem16 MOV reg16, seg-reg MOV mem16, seg-reg						
XCHG	Exchange: Switches the contents of the source and destination operands (bytes or words). When used in conjunction with the LOCK prefix, XCHG can test and set a semaphore that controls access to a resource shared by multiple processors.	XCHG dest, src	(temp) ← (dest) (dest) ← (src) (src) ← (temp)	XCHG accum, reg XCHG mem, reg XCHG reg, reg						
XLAT	Translate: Replaces a byte in the AL register with a byte from a 256-byte, user-coded translation table. Register BX is assumed to point to the beginning of the table. The byte in AL is used as an index into the table and is replaced by the byte at the offset in the table corresponding to AL's binary value. The first byte in the table has an offset of 0. For example, if AL contains 5H, and the sixth element of the translation table contains 33H, then AL will contain 33H following the instruction. XLAT is useful for translating characters from one code to another, the classic example being ASCII to EBCDIC or the reverse.	XLAT translate-table	AL ← ((BX) + (AL))	XLAT src-table						
LEA	Load Effective Address: Transfers the offset of the source operand (rather than its value) to the destination operand.	LEA dest, src	(dest) ← Offset address of mem.	LEA reg16, mem16						
LDS	Load Pointer Using DS: Transfers a 32-bit pointer variable from the source operand, which must be a memory operand, to the destination operand and register DS. The offset word of the pointer is transferred to the destination operand, which may be any 16-bit general register. The segment word of the pointer is transferred to reg. DS.	LDS dest, src	(dest) ← (EA) (DS) ← (EA + 2)	LDS reg16, mem32						
LES	Load Pointer Using ES: Transfers a 32-bit pointer variable from the source operand to the destination operand and register ES. The offset word of the pointer is transferred to the destination operand. The segment word of the pointer is transferred to reg. ES.	LES dest, src	(dest) ← (EA) (ES) ← (EA + 2)	LES reg16, mem						
Arithmetic Group of Instructions: à Instructions in this group consist of addition, subtraction, multiplication and division.										
ADD	Addition: Sums two operands, which may be bytes or words, replaces the destination operand. Both operands may be signed or unsigned binary numbers (see AAA and DAA).	ADD dest, src	(dest) ← (dest) + (src)	ADD reg, reg ADD reg, mem ADD mem, reg ADD reg, immed ADD mem, immed ADD accum, immed	C	C	C	C	C	C

Мнемо код (80C186)	Описание	Формат на командата	Действие	Операнди	O	S	Z	A	P	C
ADC	Add with Carry: Sums the operands, which may be bytes or words, adds one if CF is set and replaces the destination operand with the result. Both operands may be signed or unsigned binary numbers (see AAA and DAA). Since ADC incorporates a carry from a previous operation, it can be used to write routines to add numbers longer than 16 bits.	ADC dest, src	if (CF) = 1 then (dest) ← (dest) + (src) + 1 else (dest) ← (dest) + (src)	ADC reg, reg ADC reg, mem ADC mem, reg ADC reg, immed ADC mem, immed ADC accum, immed	C	C	C	C	C	C
SUB	Subtract: The source operand is subtracted from the destination operand, and the result replaces the destination operand. The operands may be bytes or words. Both operands may be signed or unsigned binary numbers (see AAS and DAS).	SUB dest, src	(dest) ← (dest) – (src)	SUB reg, reg SUB reg, mem SUB mem, reg SUB accum, immed SUB reg, immed SUB mem, immed	C	C	C	C	C	C
SBB	Subtract With Borrow: Subtracts the source from the destination, subtracts one if CF is set, and returns the result to the destination operand. Both operands may be bytes or words. Both operands may be signed or unsigned binary numbers (see AAS and DAS)	SBB dest, src	if (CF) = 1 then (dest) = (dest) – (src) – 1 else (dest) ← (dest) – (src)	SBB reg, reg SBB reg, mem SBB mem, reg SBB accum, immed SBB reg, immed SBB mem, immed	C	C	C	C	C	C
INC	Increment: Adds one to the destination operand. The operand may be byte or a word and is treated as an unsigned binary number (see AAA and DAA).	INC dest	(dest) ← (dest) + 1	INC reg INC mem	C	C	C	C	C	C
DEC	Decrement: Subtracts one from the destination operand. The operand may be a byte or a word and is treated as an unsigned binary number (see AAA and DAA).	DEC dest	(dest) ← (dest) – 1	DEC reg DEC mem	C	C	C	C	C	C
MUL	Multiply: Performs an unsigned multiplication of the source operand and the accumulator. If the source is a byte, then it is multiplied by register AL, and the double-length result is returned in AH and AL. If the source operand is a word, then it is multiplied by register AX, and the double-length result is returned in registers DX and AX. The operands are treated as unsigned binary numbers (see AAM). If the upper half of the result (AH for byte source, DX for word source) is nonzero, CF and OF are set; otherwise they are cleared.	MUL src	When Source Operand is a Byte: (AX) ← (AL) × (src) if (AH) = 0 then (CF) ← 0 else (CF) ← 1; (OF) ← (CF) When Source Operand is a Word: (DX:AX) ← (AX) × (src) if (DX) = 0 then (CF) ← 0 else (CF) ← 1; (OF) ← (CF)	MUL reg MUL mem	C	?	?	?	?	C
IMUL	Integer Multiply: Performs a signed multiplication of the source operand and the accumulator.	IMUL src	When Source Operand is a Byte: (AX) ← (byte-src) × (AL) if (AH) = sign-extension of (AL) then (CF) ← 0 else (CF) ← 1 (OF) ← (CF) When Source Operand is a Word: (DX:AX) ← (word-src) × (AX) if (DX) = sign-extension of (AX) then (CF) ← 0 else (CF) ← 1 (OF) ← (CF)	IMUL reg IMUL mem IMUL immed	C	?	?	?	?	C

Мнемо код (80C186)	Описание	Формат на командата	Действие	Операнди	O	S	Z	A	P	C
DIV	Unsigned Divide: Performs an unsigned division of the accumulator (and its extension) by the source operand.	DIV src	When Source Operand is a Byte: (temp) ← (byte-src) if (temp) / (AX) > FFH then (type 0 interrupt is generated) else (AL) ← (temp) / (AX) (AH) ← (temp) % (AX) When Source Operand is a Word: (temp) ← (word-src) if (temp) / (DX:AX) > FFFFH then (type 0 interrupt is generated) else (AX) ← (temp) / (DX:AX) (DX) ← (temp) % (DX:AX)	DIV reg DIV mem	?	?	?	?	?	?
IDIV	Integer Divide: Performs a signed division of the accumulator (and its extension) by the source operand.	IDIV src	When Source Operand is a Byte: (temp) ← (byte-src) if (temp) / (AX) is not correct then (type 0 interrupt is generated) else (AL) ← (temp) / (AX) (AH) ← (temp) % (AX) When Source Operand is a Word: (temp) ← (word-src) if (temp) / (DX:AX) is not correct then (type 0 interrupt is generated) else (AX) ← (temp) / (DX:AX) (DX) ← (temp) % (DX:AX)	IDIV reg IDIV mem	?	?	?	?	?	?
NEG	Negate: Subtracts the destination operand, which may be a byte or a word, from 0 and returns the result to the destination. This forms the two's complement of the number, effectively reversing the sign of an integer. If the operand is zero, its sign is not changed. Attempting to negate a byte containing –128 or a word containing – 32,768 causes no change to the operand and sets OF.	NEG dest	When Source Operand is a Byte: (dest) ← FFH – (dest) (dest) ← (dest) + 1 (affecting flags) When Source Operand is a Word: (dest) ← FFFFH – (dest) (dest) ← (dest) + 1 (affecting flags)	NEG reg NEG mem		C	C	C	C	C
CBW	Convert Byte to Word: Extends the sign of the byte in register AL throughout register AH. Use to produce a double-length (word) dividend from a byte prior to performing byte division.	CBW	if (AL) < 80H then (AH) ← 0 else (AH) ← FFH	none						
CWD	Convert Word to Double Word: Extends the sign of the word in register AX throughout register DX. Use to produce a double-length (doubleword) dividend from a word prior to performing word division.	CWD	if (AX) < 8000H then (DX) ← 0 else (DX) ← FFFFH	none						
BCD and ASCII Arithmetic: Packed BCD: à Following instructions are needed to do arithmetic on packed BCD numbers.										
DAA	Decimal Adjust for Addition: Corrects the result of previously adding two valid packed decimal operands (the destination operand must have been register AL). Changes the content of AL to a pair of valid packed decimal digits. This instruction corrects the result in AL of adding two packed BCD operands.	DAA	if ((AL) and 0FH) > 9 or (AF) = 1 then (AL) ← (AL) + 6 (AF) ← 1 if (AL) > 9FH or (CF) = 1 then (AL) ← (AL) + 60H (CF) ← 1	none	?	C	C	C	C	C

Мнемо код (80C186)	Описание	Формат на командата	Действие	Операнди	O	S	Z	A	P	C
DAS	Decimal Adjust for Subtraction: Corrects the result of a previous subtraction of two valid packed decimal operands (the destination operand must have been specified as register AL). Changes the content of AL to a pair of valid packed decimal digits.	DAS	if $((AL) \text{ and } 0FH) > 9$ or $(AF) = 1$ then $(AL) \leftarrow (AL) - 6$ $(AF) \leftarrow 1$ if $(AL) > 9FH$ or $(CF) = 1$ then $(AL) \leftarrow (AL) - 60H$ $(CF) \leftarrow 1$	none	?	C	C	C	C	C
BCD and ASCII Arithmetic: Unpacked BCD: à Following instructions are needed to do arithmetic on unpacked BCD numbers.										
AAA	ASCII Adjust After Addition: This instruction corrects the result in AL of adding two unpacked BCD digits or two ASCII digits.	AAA	if $((AL) \text{ and } 0FH) > 9$ or $(AF) = 1$ then $(AL) \leftarrow (AL) + 6$ $(AH) \leftarrow (AH) + 1$ $(AF) \leftarrow 1$ $(CF) \leftarrow (AF)$ $(AL) \leftarrow (AL) \text{ and } 0FH$	none	?	?	?	C	?	C
AAS	ASCII Adjust AL After Subtraction - This instruction corrects the result in AL of subtracting two unpacked BCD digits or two ASCII digits.	AAS	if $((AL) \text{ and } 0FH) > 9$ or $(AF) = 1$ then $(AL) \leftarrow (AL) - 6$ $(AH) \leftarrow (AH) - 1$ $(AF) \leftarrow 1$ $(CF) \leftarrow (AF)$ $(AL) \leftarrow (AL) \text{ and } 0FH$	none	?	?	?	C	?	C
AAM	ASCII Adjust for Multiply: Corrects the result of a previous multiplication of two valid unpacked decimal operands. A valid 2-digit unpacked decimal number is derived from the content of AH and AL and is returned to AH and AL. The high-order half-bytes of the multiplied operands must have been 0H for AAM to produce a correct result.	AAM	$(AH) \leftarrow (AL) / 0AH$ $(AL) \leftarrow (AL) \% 0AH$	none	?	C	C	?	C	?
AAD	ASCII Adjust for Division: Modifies the numerator in AL before dividing two valid unpacked decimal operands so that the quotient produced by the division will be a valid unpacked decimal number. AH must be zero for the subsequent DIV to produce the correct result. The quotient is returned in AL, and the remainder is returned in AH; both highorder half-bytes are zeroed.	AAD	$(AL) \leftarrow (AH) \times 0AH + (AL)$ $(AH) \leftarrow 0$	none	?	C	C	?	C	?
Comparison Instructions: à Compares the operands by subtracting them. Result is not stored anywhere only the flag bits change.										
CMP	Compare two operands: Compares the operands by subtracting them. Result is not stored anywhere only the flag bits change.	CMP dest, src	$(dest) - (src) =$ affects the status flags.	CMP reg, reg CMP reg, mem CMP mem, reg CMP reg, immed CMP mem, immed CMP accum, immed	C	C	C	C	C	C
CMPS	Compare String Operands: Subtracts the destination byte or word from the source byte or word. The destination byte or word is addressed by the destination index (DI) register and the source byte or word is addresses by the source index (SI) register. CMPS updates the flags to reflect the relationship of the destination element to the source element but does not alter either operand and updates SI and DI to point to the next string element.	CMPS dest-string, src-string	$(dest-string) - (src-string)$ if $(DF) = 0$ then $(SI) \leftarrow (SI) + DELTA$ $(DI) \leftarrow (DI) + DELTA$ else $(SI) \leftarrow (SI) - DELTA$ $(DI) \leftarrow (DI) - DELTA$	CMP dest-string, src-string CMP reg (repeat) dest-string, src-string	C	C	C	C	C	C
Logic Group of Instructions: à Instructions in this group performs AND, OR, XOR and NOT logic operations.										

Мнемо код (80C186)	Описание	Формат на командата	Действие	Операнди	O	S	Z	A	P	C
AND	Logical AND: Performs the logical "and" of the two operands (byte or word) and returns the result to the destination operand. A bit in the result is set if both corresponding bits of the original operands are set; otherwise the bit is cleared.	AND dest, src	(dest) ← (dest) and (src) (CF) ← 0 (OF) ← 0	AND reg, reg AND reg, mem AND mem, reg AND reg, immed AND mem, immed AND accum, immed	0	C	C	?	C	0
OR	Logical OR: Performs the logical "inclusive or" of the two operands (bytes or words) and returns the result to the destination operand. A bit in the result is set if either or both corresponding bits in the original operands are set; otherwise the result bit is cleared.	OR dest,src	(dest) ← (dest) or (src) (CF) ← 0 (OF) ← 0	OR reg, reg OR reg, mem OR mem, reg OR accum, immed OR reg, immed OR mem, immed	0	C	C	?	C	0
XOR	Exclusive Or: Performs the bitwise, logical "exclusive or" of the two operands and returns the result to the destination operand. A bit in the result is set if the corresponding bits of the original operands contain opposite values (one is set, the other is cleared); otherwise the result bit is cleared.	XOR dest, src	(dest) ← (dest) xor (src) (CF) ← 0 (OF) ← 0	XOR reg, reg XOR reg, mem XOR mem, reg XOR accum, immed XOR reg, immed XOR mem, immed	0	C	C	?	C	0
NOT	Logical Not: Inverts the bits (forms the one's complement) of the byte or word operand.	NOT dest	When Source Operand is a Byte: (dest) ← FFH – (dest) When Source Operand is a Word: (dest) ← FFFFH – (dest)	NOT reg NOT mem	C	C	C	?	C	C
TEST	Test: Performs the logical "and" of the two operands (bytes or words), updates the flags, but does not return the result, i.e., neither operand is changed. If a TEST instruction is followed by a JNZ (jump if not zero) instruction, the jump will be taken if there are any corresponding one bits in both operands.	TEST dest, src	(dest) and (src) (CF) ← 0 (OF) ← 0	TEST reg, reg TEST reg, mem TEST accum, immed TEST reg, immed TEST mem, immed	0	C	C	?	C	0
NOP	No Operation: Causes the CPU to do nothing.	NOP	None	none						
Shift and Rotate Instructions: à Shift and Rotate instructions manipulate binary numbers at the binary bit level.										
SHL SAL	Shift Logical Left: Shift Arithmetic Left: Shifts the destination byte or word left by the number of bits specified in the count operand. Zeros are shifted in on the right. If the sign bit retains its original value, then OF is cleared.	SHL dest, count SAL dest, count	(temp) ← count do while (temp) ≠ 0 (CF) ← high-order bit of (dest) (dest) ← (dest) × 2 (temp) ← (temp) – 1 if count = 1	SHL reg, n SAL reg, n SHL mem, n SAL mem, n SHL reg, CL SAL reg, CL SHL mem, CL SAL mem, CL	C	C	C	?	C	C
SHR	Shift Logical Right: Shifts the bits in the destination operand (byte or word) to the right by the number of bits specified in the count operand. Zeros are shifted in on the left. If the sign bit retains its original value, then OF is cleared.	SHR dest, count	(temp) ← count do while (temp) ≠ 0 (CF) ← low-order bit of (dest) (dest) ← (dest) / 2 (temp) ← (temp) – 1	SHR reg, n SHR mem, n SHR reg, CL SHR mem, CL	C	C	C	?	C	0
SAR	Shift Arithmetic Right: Shifts the bits in the destination operand (byte or word) to the right by the number of bits specified in the count operand. Bits equal to the original high-order (sign) bit are shifted in on the left, preserving the sign of the original value. Note that SAR does not produce the same result as the dividend of an "equivalent" IDIV instruction if the destination operand is negative and 1 bits are shifted out. For example, shifting –5 right by one bit yields –3, while integer division –5 by 2 yields –2. The difference in the instructions is that IDIV truncates all numbers toward zero, while SAR truncates positive numbers toward zero and negative numbers toward negative infinity.	SAR dest, count	(temp) ← count do while (temp) ≠ 0 (CF) ← low-order bit of (dest) (dest) ← (dest) / 2 (temp) ← (temp) – 1 if count = 1 then if high-order bit of (dest) ≠ next-to-high-order bit of (dest)	SAR reg, n SAR mem, n SAR reg, CL SAR mem, CL	C	C	C	?	C	C

Мнемо код (80C186)	Описание	Формат на командата	Действие	Операнди	O	S	Z	A	P	C
ROL	Rotate Left: Rotates the destination byte or word left by the number of bits specified in the count operand.	ROL dest, count	(temp) ← count do while (temp) ≠ 0 (CF) ← high-order bit of (dest) (dest) ← (dest) × 2 + (CF) (temp) ← (temp) – 1	ROL reg, n ROL mem, n ROL reg, CL ROL mem CL	C					C
ROR	Rotate Right: Operates similar to ROL except that the bits in the destination byte or word are rotated right instead of left.	ROR dest, count	(temp) ← count do while (temp) ≠ 0 (CF) ← low-order bit of (dest) (dest) ← (dest) / 2 high-order bit of (dest) ← (CF) (temp) ← (temp) – 1	ROR reg, n ROR mem, n ROR reg, CL ROR mem, CL	C					C
RCL	Rotate Through Carry Left: Rotates the bits in the byte or word destination operand to the left by the number of bits specified in the count operand. The carry flag (CF) is treated as "part of" the destination operand; that is, its value is rotated into the loworder bit of the destination, and itself is replaced by the high-order bit of the destination.	RCL dest, count	(temp) ← count do while (temp) ≠ 0 (tmpcf) ← (CF) (CF) ← high-order bit of (dest) (dest) ← (dest) × 2 + (tmpcf) (temp) ← (temp) – 1	RCL reg, n RCL mem, n RCL reg, CL RCL mem, CL	C					C
RCR	Rotate Through Carry Right: Operates exactly like RCL except that the bits are rotated right instead of left.	RCR dest, count	(temp) ← count do while (temp) ≠ 0 (tmpcf) ← (CF) (CF) ← low-order bit of (dest) (dest) ← (dest) / 2 high-order bit of (dest) ← (tmpcf) (temp) ← (temp) – 1	RCR reg, n RCR mem, n RCR reg, CL RCR mem, CL	C					C
Flag Control Instructions: à Affects carry, direction, and interrupt flags.										
CLC	Clear Carry Flag: Zeroes the carry flag (CF) and affects no other flags. Useful in conjunction with the rotate through carry left (RCL) and the rotate through carry right (RCR) instructions.	CLC	(CF) ← 0.	none						0
CLD	Clear Direction Flag: Zeroes the direction flag (DF) causing the string instructions to autoincrement the source index (SI) and/or destination index (DI) registers.	CLD	(DF) ← 0	none						
CLI	Clear Interrupt Flag: Zeroes the interrupt-enable flag (IF). When the interrupt-enable flag is cleared, the 8086 and 8088 do not recognize an external interrupt request that appears on the INTR line; in other words maskable interrupts are disabled. A non-maskable interrupt appearing on NMI line, however, is honored, as is a software interrupt.	CLI	(IF) ← 0	none						
CMC	Complement Carry Flag: Toggles complement carry flag (CF) to its opposite state and affects no other flags.	CMC	if (CF) = 0 then (CF) ← 1 else (CF) ← 0	none						C
STC	Set Carry Flag Sets CF to 1.	STC	(CF) ← 1	none						1
STD	Set Direction Flag: Sets DF to 1 causing the string instructions to auto-decrement the SI and/or DI index registers.	STD	(DF) ← 1	none						

Мнемо код (80C186)	Описание	Формат на командата	Действие	Операнди	O	S	Z	A	P	C
STI	Set Interrupt-enable Flag: Sets IF to 1, enabling processor recognition of maskable interrupt requests appearing on the INTR line. Note however, that a pending interrupt will not actually be recognized until the instruction following STI has executed.	STI	$(IF) \leftarrow 1$	none						
SAHF	Store Register AH Into Flags: Transfers bits 7, 6, 4, 2, and 0 from register AH into SF, ZF, AF, PF, and CF, respectively, replacing whatever values these flags previously had.	SAHF	$(SF):(ZF):X:(AF):X:(PF):X:(CF) \leftarrow (AH)$	none		C	C	C	C	C
LAHF	Load Register AH From Flags: Copies SF, ZF, AF, PF and CF (the 8080/8085 flags) into bits 7, 6, 4, 2 and 0, respectively, of register AH. The content of bits 5, 3, and 1 are undefined. LAHF is provided primarily for converting 8080/8085 assembly language programs to run on an 8086 or 8088.	LAHF	$(AH) \leftarrow (SF):(ZF):X:(AF):X:(PF):X:(CF)$	none						
Jump Instructions: These instructions allow the programmer to skip sections of a program and branch to any part of the memory for the next instruction.										
Jump Instructions: à Unconditional Jumps (JMP)										
JMP	Jump Unconditionally: Transfers control to the target location. Three types of unconditional jump instructions are available to the microprocessor: - Short jump - Near jump - Far jump	JMP target	if Inter-segment then $(CS) \leftarrow SEG$ $(IP) \leftarrow dest$	JMP short-label JMP near-label JMP far-label JMP memptr JMP regptr						
Jump Instructions: à Conditional Jumps → Conditional jumps are always short jumps in the 8086/8088 microprocessors. So, the limits of the range of jumps are within +7Fh bytes and -7Fh bytes from the location following the conditional jump. → S, Z, C, P, and O flags are used to test the condition and execute the jump. → Operation of most conditional jump instructions is straight forward because they often test just one flag bit, although some test more than one. → The term greater and lesser than refers to signed numbers jump instructions. Z, S and O flags are checked before executing jump. → The term above and below refers to unsigned numbers jump instructions. Z and C flags are checked before executing jump.										
Jump Instructions: à Conditional Jumps Unsigned data										
JE JZ	Jump on Equal: Jump on Zero: Transfers control to the target location if the condition tested ($ZF = 1$) is true.	JE disp8 JZ disp8	if $(ZF) = 1$ then $(IP) \leftarrow (IP) + disp8$ (sign-ext to 16 bits)	JE short-label JZ short-label						
JNE JNZ	Jump on Not Equal: Jump on Not Zero: Transfers control to the target location if the tested condition ($ZF = 0$) is true.	JNE disp8 JNZ disp8	if $(ZF) = 0$ then $(IP) \leftarrow (IP) + disp8$ (sign-ext to 16 bits)	JNE short-label JNZ short-label						
JA JNBE	Jump on Above: Jump on Not Below or Equal: Transfers control to the target location if the tested condition ($(CF=0)$ or $(ZF=0)$) is true.	JA disp8 JNBE disp8	if $((CF) = 0)$ or $((ZF) = 0)$ then $(IP) \leftarrow (IP) + disp8$ (sign-ext to 16 bits)	JA short-label JNBE short-label						

Мнемо код (80C186)	Описание	Формат на командата	Действие	Операнди	O	S	Z	A	P	C
JAE JNB	Jump on Above or Equal: Jump on Not Below: Transfers control to the target location if the tested condition (CF = 0) is true.	JAE disp8 JNB disp8	if (CF) = 0 then (IP) ← (IP) + disp8 (sign-ext to 16 bits)	JAE short-label JNB short-label						
JB JNAE	Jump on Below: Jump on Not Above or Equal: Transfers control to the target location if the tested condition (CF = 1) is true.	JB disp8 JNAE disp8	if (CF) = 1 then (IP) ← (IP) + disp8 (sign-ext to 16 bits)	JB short-label JNAE short-label						
JBE JNA	Jump on Below or Equal: Jump on Not Above: Transfers control to the target location if the tested condition ((C = 1) or (ZF=1)) is true.	JBE disp8 JNA disp8	if ((CF) = 1) or ((ZF) = 1) then (IP) ← (IP) + disp8 (sign-ext to 16 bits)	JBE short-label JNA short-label						
Jump Instructions: à Conditional Jumps Signed data										
JG JNLE	Jump on Greater Than: Jump on Not Less Than or Equal: Transfers control to the target location if the condition tested (SF = OF) and (ZF=0) is true.	JG disp8 JNLE disp8	if ((SF) = (OF)) and ((ZF) = 0) then (IP) ← (IP) + disp8 (sign-ext to 16 bits)	JG short-label JNLE short-label						
JGE JNL	Jump on Greater Than or Equal: Jump on Not Less Than: Transfers control to the target location if the condition tested (SF=OF) is true.	JGE disp8 JNL disp8	if (SF) = (OF) then (IP) ← (IP) + disp8 (sign-ext to 16 bits)	JGE short-label JNL short-label						
JL JNGE	Jump on Less Than: Jump on Not Greater Than or Equal: Transfers control to the target location if the condition tested (SF≠OF) is true.	JL disp8 JNGE disp8	if (SF) ≠ (OF) then (IP) ← (IP) + disp8 (sign-ext to 16 bits)	JL short-label JNGE short-label						
JLE JNG	Jump on Less Than or Equal: Jump on Not Greater Than: Transfers control to the target location If the condition tested ((SF≠OF) or (ZF=0)) is true.	JLE disp8 JNG disp8	if ((SF) ≠ (OF)) or ((ZF) = 1) then (IP) ← (IP) + disp8 (sign-ext to 16 bits)	JLE short-label JNG short-label						
Jump Instructions: à Conditional Jumps Special arithmetic check										
JS	Jump on Sign: Transfers control to the target location if the tested condition (SF = 1) is true.	JS disp8	if (SF) = 1 then (IP) ← (IP) + disp8 (sign-ext to 16 bits)	JS short-label						
JNS	Jump on Not Sign: Transfers control to the target location if the tested condition (SF = 0) is true.	JNS disp8	if (SF) = 0 then (IP) ← (IP) + disp8 (sign-ext to 16 bits)	JNS short-label						
JC	Jump on Carry: Transfers control to the target location if the tested condition (CF=1) is true.	JC disp8	if (CF) = 1 then (IP) ← (IP) + disp8 (sign-ext to 16 bits)	JC short-label						
JNC	Jump on Not Carry: Transfers control to the target location if the tested condition (CF=0) is true.	JNC disp8	if (CF) = 0 then (IP) ← (IP) + disp8 (sign-ext to 16 bits)	JNC short-label						
JO	Jump on Overflow: Transfers control to the target location if the tested condition (OF = 1) is true.	JO disp8	if (OF) = 1 then (IP) ← (IP) + disp8 (sign-ext to 16 bits)	JO short-label						
JNO	Jump on Not Overflow: Transfers control to the target location if the tested condition (OF = 0) is true.	JNO disp8	if (OF) = 0 then (IP) ← (IP) + disp8 (sign-ext to 16 bits)	JNO short-label						

Мнемо код (80C186)	Описание	Формат на командата	Действие	Операнди	O	S	Z	A	P	C
Jump Instructions: à Conditional Jumps Other condition										
JNP JPO	Jump on Not Parity: Jump on Parity Odd: Transfers control to the target location if the tested condition (PF=0) is true.	JNO disp8 JPO disp8	if (PF) = 0 then (IP) ← (IP) + disp8 (sign-ext to 16 bits)	JNO short-label JPO short-label						
JP JPE	Jump on Parity: Jump on Parity Equal: Transfers control to the target location if the tested condition (PF = 1) is true.	JP disp8 JPE disp8	if (PF) = 1 then (IP) ← (IP) + disp8 (sign-ext to 16 bits)	JP short-label JPE short-label						
Jump Instructions: à Conditional Jumps Condition is a register										
JCXZ	Jump if CX Zero: Transfers control to the target location loop to bypass the loop if CX has a zero value, i.e., to execute the loop zero times.	JCXZ disp8	if (CX) = 0 then (IP) ← (IP) + disp8 (sign-ext to 16 bits)	JCXZ short-label						
Subroutine and Interrupt Handling Instructions: à CALL and RET instructions together provide the mechanism for calling a subroutine into operation and returning control back to the main program at its completion.										
CALL	Call Procedure: Activates an out-of-line procedure, saving information on the stack to permit a RET (return) instruction in the procedure to transfer control back to the instruction following the CALL. The assembler generates a different type of CALL instruction depending on whether the programmer has defined the procedure name as NEAR or FAR.	CALL procedure-name	if Inter-segment then (SP) ← (SP) – 2 ((SP) + 1:(SP)) ← (CS) (CS) ← SEG (SP) ← (SP) – 2 ((SP) + 1:(SP)) ← (IP) (IP) ← dest	CALL near-proc CALL far-proc CALL memptr16 CALL regptr16 CALL memptr32						
RET	Return: Transfers control from a procedure back to the instruction following the CALL that activated the procedure. The assembler generates an intrasegment RET if the programmer has defined the procedure near, or an intersegment RET if the procedure has been defined as far. RET pops the word at the top of the stack (pointed to by register SP) into the instruction pointer and increments SP by two. If RET is intersegment, the word at the new top of stack is popped into the CS register, and SP is again incremented by two. If an optional pop value has been specified, RET adds that value to SP.	RET optional-pop-value	(IP) ← ((SP) = 1:(SP)) (SP) ← (SP) + 2 if inter-segment then (CS) ← ((SP) + 1:(SP)) (SP) ← (SP) + 2 if add imm8 to SP then (SP) ← (SP) + data	RET imm8						
INT	Interrupt: Activates the interrupt procedure specified by the interrupt-type operand. Decrements the stack pointer by two, pushes the flags onto the stack, and clears the trap (TF) and interrupt-enable (IF) flags to disable single-step and maskable interrupts. The flags are stored in the format used by the PUSHF instruction. SP is decremented again by two, and the CS register is pushed onto the stack. The address of the interrupt pointer is calculated by multiplying interrupttype by four; the second word of the interrupt pointer replaces CS. SP again is decremented by two, and IP is pushed onto the stack and is replaced by the first word of the interrupt pointer. If interrupt-type = 3, the assembler generates a short (1 byte) form of the instruction, known as the breakpoint interrupt.	INT interrupt-type	(SP) ← (SP) – 2 ((SP) + 1:(SP)) ← FLAGS (IF) ← 0 (TF) ← 0 (SP) ← (SP) – 2 ((SP) + 1:(SP)) ← (CS) (CS) ← (interrupt-type × 4 + 2) (SP) ← (SP) – 2 ((SP) + 1:(SP)) ← (IP) (IP) ← (interrupt-type × 4)	INT imm8						

Мнемо код (80C186)	Описание	Формат на командата	Действие	Операнди	O	S	Z	A	P	C
INTO	Interrupt on Overflow: Generates a software interrupt if the overflow flag (OF) is set; otherwise control proceeds to the following instruction without activating an interrupt procedure. INTO addresses the target interrupt procedure (its type is 4) through the interrupt pointer at location 10H; it clears the TF and IF flags and otherwise operates like INT. INTO may be written following an arithmetic or logical operation to activate an interrupt procedure if overflow occurs.	INTO	if (OF) = 1 then (SP) ← (SP) – 2 ((SP) + 1:(SP)) ← FLAGS (IF) ← 0 (TF) ← 0 (SP) ← (SP) – 2 ((SP) + 1:(SP)) ← (CS) (CS) ← (12H) (SP) ← (SP) – 2 ((SP) + 1:(SP)) ← (IP) (IP) ← (10H)	none						
IRET	Interrupt Return: Transfers control back to the point of interruption by popping IP, CS, and the flags from the stack. IRET thus affects all flags by restoring them to previously saved values. IRET is used to exit any interrupt procedure, whether activated by hardware or software.	IRET	(IP) ← ((SP) + 1:(SP)) (SP) ← (SP) + 2 (CS) ← ((SP) + 1:(SP)) (SP) ← (SP) + 2 FLAGS ← ((SP) + 1:(SP)) (SP) ← (SP) + 2	none	C	C	C	C	C	C
LOOP Instructions: Loop instructions are used to repeatedly execute a piece of code/instructions.										
LOOP	Loop: Decrements CX by 1 and transfers control to the target location if CX is not 0; otherwise the instruction following LOOP is executed.	LOOP disp8	Loop until CX = 0: (CX) ← (CX) – 1 if (CX) ≠ 0 then (IP) ← (IP) + disp8 (sign-ext to 16 bits)	LABEL : short label.						
LOOPE / LOOPZ	Loop While Equal: Loop While Zero: Decrements CX by 1 and transfers control to the target location if CX is not 0 and if ZF is set; otherwise the next sequential instruction is executed.	LOOPE disp8 LOOPZ disp8	Loop while ZF = 1: (CX) ← (CX) – 1 if (ZF) = 1 and (CX) ≠ 0 then (IP) ← (IP) + disp8 (sign-ext to 16 bits)	LOOPE short-label LOOPZ short-label						
LOOPNE / LOOPNZ	Loop While Not Equal: Loop While Not Zero: Decrements CX by 1 and transfers control to the target location if CX is not 0 and if ZF is clear; otherwise the next sequential instruction is executed.	LOOPNE disp8 LOOPNZ disp8	Loop while ZF = 0: (CX) ← (CX) – 1 if (ZF) = 0 and (CX) ≠ 0 then (IP) ← (IP) + disp8 (sign-ext to 16 bits)	LOOPNE short-label LOOPNZ short-label						
Stack Operations Instructions: a PUSH and POP instructions store data onto and retrieve data from the LIFO stack memory.										
PUSH	Push: Decrements SP by two and then transfers a word from the source operand to the top of stack now pointed to by SP.	PUSH src	(SP) ← (SP) – 2 ((SP) + 1:(SP)) ← (src)	PUSH reg PUSH seg-reg (CS legal) PUSH mem						
PUSHA	Push All: Pushes all data, pointer, and index registers onto the stack. The order in which the registers are saved is: AX, CX, DX, BX, SP, BP, SI, and DI. The SP value pushed is the SP value before the first register (AX) is pushed.	PUSHA	temp ← (SP) (SP) ← (SP) – 2 ; ((SP) + 1:(SP)) ← (AX) (SP) ← (SP) – 2 ; ((SP) + 1:(SP)) ← (CX) (SP) ← (SP) – 2 ; ((SP) + 1:(SP)) ← (DX) (SP) ← (SP) – 2 ; ((SP) + 1:(SP)) ← (BX) (SP) ← (SP) – 2 ; ((SP) + 1:(SP)) ← (temp) (SP) ← (SP) – 2 ; ((SP) + 1:(SP)) ← (BP) (SP) ← (SP) – 2 ; ((SP) + 1:(SP)) ← (SI) (SP) ← (SP) – 2 ; ((SP) + 1:(SP)) ← (DI)	none						

Мнемо код (80C186)	Описание	Формат на командата	Действие	Операнди	O	S	Z	A	P	C
PUSHF	Push Flags: Decrements SP by two and then transfers all flags to the word at the top of stack pointed to by SP.	PUSHF	$(SP) \leftarrow (SP) - 2$ $((SP) + 1:(SP)) \leftarrow \text{Flags}$	none						
POP	Pop: Transfers the word at the current top of stack (pointed to by SP) to the destination operand and then increments SP by two to point to the new top of stack.	POP dest	$(\text{dest}) \leftarrow ((SP) + 1:(SP))$ $(SP) \leftarrow (SP) + 2$	POP reg POP seg-reg (CS illegal) POP mem						
POPA	Pop All: Pops all data, pointer, and index registers off of the stack. The SP value popped is discarded.	POPA	$(DI) \leftarrow ((SP) + 1:(SP)) ; (SP) \leftarrow (SP) + 2$ $(SI) \leftarrow ((SP) + 1:(SP)) ; (SP) \leftarrow (SP) + 2$ $(BP) \leftarrow ((SP) + 1:(SP)) ; (SP) \leftarrow (SP) + 2$ $(BX) \leftarrow ((SP) + 1:(SP)) ; (SP) \leftarrow (SP) + 2$ $(DX) \leftarrow ((SP) + 1:(SP)) ; (SP) \leftarrow (SP) + 2$ $(CX) \leftarrow ((SP) + 1:(SP)) ; (SP) \leftarrow (SP) + 2$ $(AX) \leftarrow ((SP) + 1:(SP)) ; (SP) \leftarrow (SP) + 2$	none						
POPF	Pop Flags: Transfers specific bits from the word at the current top of stack (pointed to by register SP) into the 8086/8088 flags, replacing whatever values the flags previously contained. SP is then incremented by two to point to the new top of stack.	POPF	$\text{Flags} \leftarrow ((SP) + 1:(SP))$ $(SP) \leftarrow (SP) + 2$	none	C	C	C	C	C	C
String Handling Instructions: à These instructions are used to deal with string data.										
LODS	Load String (Byte or Word): Transfers the byte or word string element addressed by SI to register AL or AX and updates SI to point to the next element in the string. This instruction is not ordinarily repeated since the accumulator would be overwritten by each repetition, and only the last element would be retained.	LODS src-string	When Source Operand is a Byte: $(AL) \leftarrow (\text{src-string})$ if $(DF) = 0$ then $(SI) \leftarrow (SI) + \text{DELTA}$ else $(SI) \leftarrow (SI) - \text{DELTA}$ When Source Operand is a Word: $(AX) \leftarrow (\text{src-string})$ if $(DF) = 0$ then $(SI) \leftarrow (SI) + \text{DELTA}$ else $(SI) \leftarrow (SI) - \text{DELTA}$	LODS src-string LODS (repeat) src-string						
STOS	Store (Byte or Word) String: Transfers a byte or word from register AL or AX to the string element addressed by DI and updates DI to point to the next location in the string. As a repeated operation.	STOS dest-string	When Source Operand is a Byte: $(\text{DEST}) \leftarrow (AL)$ if $(DF) = 0$ then $(DI) \leftarrow (DI) + \text{DELTA}$ else $(DI) \leftarrow (DI) - \text{DELTA}$ When Source Operand is a Word: $(\text{DEST}) \leftarrow (AX)$ if $(DF) = 0$ then $(DI) \leftarrow (DI) + \text{DELTA}$ else $(DI) \leftarrow (DI) - \text{DELTA}$	STOS dest-string STOS (repeat) dest-string						
MOVS	Move String: Transfers a byte or a word from the source string (addressed by SI) to the destination string (addressed by DI) and updates SI and DI to point to the next string element. When used in conjunction with REP, MOVS performs a memory-to-memory block transfer.	MOVS dest-string, src-string	$(\text{dest-string}) \leftarrow (\text{src-string})$	MOVS dest-string, src-string MOVS (repeat) dest-string, src-string						

Мнемо код (80C186)	Описание	Формат на командата	Действие	Операнди	O	S	Z	A	P	C
INS	In String: Performs block input from an I/O port to memory. The port address is placed in the DX register. The memory address is placed in the DI register. This instruction uses the ES register (which cannot be overridden). After the data transfer takes place, the DI register increments or decrements, depending on the value of the direction flag (DF). The DI register changes by 1 for byte transfers or 2 for word transfers.	INS dest-string, port	$(dest) \leftarrow (src)$	INS dest-string, port INS (repeat) dest-string, port						
OUTS	Out String: Performs block output from memory to an I/O port. The port address is placed in the DX register. The memory address is placed in the SI register. This instruction uses the DS segment register, but this may be changed with a segment override instruction. After the data transfer takes place, the pointer register (SI) increments or decrements, depending on the value of the direction flag (DF). The pointer register changes by 1 for byte transfers or 2 for word transfers.	OUTS port, src_string	$(dst) \leftarrow (src)$	OUTS port, src_string OUTS (repeat) port, src_string						
SCAS	Scan String: Subtracts the destination string element (byte or word) addressed by DI from the content of AL (byte string) or AX (word string) and updates the flags, but does not alter the destination string or the accumulator. SCAS also updates DI to point to the next string element and AF, CF, OF, PF, SF and ZF to reflect the relationship of the scan value in AL/AX to the string element. If SCAS is prefixed with REPE or REPZ, the operation is interpreted as "scan while not end-ofstring (CX not 0) and string-element = scan-value (ZF = 1)." This form may be used to scan for departure from a given value. If SCAS is prefixed with REPNE or REPNZ, the operation is interpreted as "scan while not end-ofstring (CX not 0) and string-element is not equal to scan-value (ZF = 0)."	SCAS dest-string	When Source Operand is a Byte: $(AL) - (\text{byte-string})$ if $(DF) = 0$ then $(DI) \leftarrow (DI) + \text{DELTA}$ else $(DI) \leftarrow (DI) - \text{DELTA}$ When Source Operand is a Word: $(AX) - (\text{word-string})$ if $(DF) = 0$ then $(DI) \leftarrow (DI) + \text{DELTA}$ else $(DI) \leftarrow (DI) - \text{DELTA}$	SCAS dest-string SCAS (repeat) dest-string	C	C	C	C	C	C
REP REPE REPZ REPNE REPNZ	Repeat: Repeat While Equal: Repeat While Zero: Repeat While Not Equal: Repeat While Not Zero: Controls subsequent string instruction repetition. The different mnemonics are provided to improve program clarity. REP is used in conjunction with the MOVS (Move String) and STOS (Store String) instructions and is interpreted as "repeat while not end-of-string" (CX not 0).	REP REPE REPZ REPNE REPNZ	do while $(CX) \neq 0$ service pending interrupts (if any) execute primitive string Operation in succeeding byte $(CX) \leftarrow (CX) - 1$ if primitive operation is CMPB, CMPW, SCAB, or SCAW and $(ZF) \neq 0$ then exit from while loop	none						
Isolated I/O Instructions: à IN and OUT instructions transfer data between the AL, AX registers of microprocessor and I/O devices.										
IN	Input from Port: Transfers a byte or a word from an input port to the AL register or the AX register, respectively. The port number may be specified either with an immediate byte constant, allowing access to ports numbered 0 through 255, or with a number previously placed in the DX register, allowing variable access (by changing the value in DX) to ports numbered from 0 through 65,535.	IN acc , port	When Source Operand is a Byte: $(AL) \leftarrow (\text{port})$ When Source Operand is a Word: $(AX) \leftarrow (\text{port})$	IN AL, immed8 IN AX, DX						
OUT	Output: Transfers a byte or a word from the AL register or the AX register, respectively, to an output port. The port number may be specified either with an immediate byte constant, allowing access to ports numbered 0 through 255, or with a number previously placed in register DX, allowing variable access (by changing the value in DX) to ports numbered from 0 through 65,535.	OUT port, accumulator	$(dest) \leftarrow (src)$	OUT immed8, AL OUT DX, AX						
Instructions to bus control										

Мнемо код (80C186)	Описание	Формат на командата	Действие	Операнди	O	S	Z	A	P	C
ESC	Escape: Provides a mechanism by which other processors (coprocessors) may receive their instructions from the 8086 or 8088 instruction stream and make use of the 8086 or 8088 addressing modes. The CPU (8086 or 8088) does a no operation (NOP) for the ESC instruction other than to access a memory operand and place it on the bus.	ESC	if mod $\neq$ 11 then data bus $\leftarrow$ (EA)	ESC immed, mem ESC immed, reg						
HLT	Halt: Causes the CPU to enter the halt state. The processor leaves the halt state upon activation of the RESET line, upon receipt of a non-maskable interrupt request on NMI, or upon receipt of a maskable interrupt request on INTR (if interrupts are enabled).	HLT	None	none						
LOCK	Lock the Bus: Causes the 8088 (configured in maximum mode) to assert its bus LOCK signal while the following instruction executes. The instruction most useful in this context is an exchange register with memory. The LOCK prefix may be combined with the segment override and/or REP prefixes.	LOCK	none	none						
WAIT	Wait: Causes the CPU to enter the wait state while its test line is not active.	WAIT	None	none						